

PARTICIPANT GUIDE

The Verified Build Hackathon

Build verifiable software, end-to-end — from prompt, to test suite, to working solution — in three hours.

FORMAT

2–3 hour sprint

AUDIENCE

Full-stack engineers (2+ yrs)

OUTPUT

A self-validating `app/` bundle

Welcome. This is not a typical "build an app" competition. You will be doing something more interesting: producing a **fully verified, reproducible software task** end-to-end — the same kind of artifact that powers modern code-evaluation pipelines and AI training data.

Every participant ships three things in lockstep: a **structured prompt**, a **functional test suite**, and a **working solution (the "Golden Patch")** that provably satisfies the prompt and passes the suite inside a sealed Docker environment.

This document is your single source of truth for the event.

Table of Contents

1. The Challenge in One Paragraph
2. Who This Is For
3. The Workflow — Six Stages
4. Step 0 — Reading the Seed Task
5. Step 1 — Prompt Engineering
6. Step 2 — Writing the Test Suite (TDD)
7. Step 3 — Docker Environment & Test Runner
8. Step 4 — Building the Golden Patch
9. Step 5 — Validation Script
10. Submission — The `app/` Folder
11. Tech Stack Compatibility Guide
12. FAQ

1. The Challenge in One Paragraph

You will be given a **seed task** — a short, freelance-style brief written like a real client request (e.g. *"Build me a website that shows my restaurant menu and hours"*). Your job is to:

1. Transform that vague brief into a **deterministic, structured prompt** that tells an AI agent exactly what to build.
2. Write a **functional unit test suite** that proves the requirements are satisfied — without locking the agent into one specific implementation.
3. Build the **Golden Patch** — a real, working codebase that satisfies the prompt and passes every test.
4. Run the **validation script** to prove that an empty repo fails all tests, and your Golden Patch passes all tests.
5. Submit the resulting `app/` folder.

The output of this hackathon is not a single "demo" — it is a **reproducible, black-box-verifiable software unit**. That is what makes it hard, and that is what makes it interesting.

2. Who This Is For

You should be comfortable with full-stack development, basic Docker usage, and writing tests in at least one mainstream language (Python, Node/TS, etc.). You do **not** need to be an expert at prompt engineering — that is exactly the skill this hackathon is designed to build.

Most participants find the *coding* part easy. The hard part is **writing a prompt so unambiguous that any reasonable AI agent — or any reasonable developer — will produce a solution that passes your tests**. That is a real, transferable skill, and it's the centerpiece of this event.

3. The Workflow — Six Stages

You will move through six stages. Stay in order — each stage feeds the next.

Step 0 → Read & understand the seed task
Step 1 → Rewrite into a deterministic structured prompt
Step 2 → Write functional unit tests (TDD – before you build!)
Step 3 → Configure Docker, run.sh, parsing.py
Step 4 → Build the Golden Patch (with help from a coding agent)
Step 5 → Run the validation script → produce before.json + after.json

Suggested time budget for a 2–3 hour run:

Stage	Suggested Time
Step 0 — Read	5–10 min
Step 1 — Prompt	30–45 min
Step 2 — Tests	30–45 min
Step 3 — Docker	10–15 min
Step 4 — Golden Patch	45–60 min
Step 5 — Validation + Submit	15–20 min

HEADS UP

If you find yourself burning more than 45 minutes on the prompt, ship what you have and move on. The *cycle* matters more than perfection on any single step.

4. Step 0 — Reading the Seed Task

The seed task will give you four fields:

- **Task Type** — the domain (CLI tool, web app, API, data pipeline, etc.)
- **Coding Language** — either a fixed language, or "Any" (you choose)
- **Short Description** — global constraints (always preserved, see below)
- **Task Description** — the actual freelance brief

Global Constraints (always apply)

These are the rules every solution must obey, even when the seed task doesn't say so:

- The solution must run **fully offline**. No live API calls, no dataset downloads at runtime.
- Any required dataset must be **bundled into the repository** (e.g. a local JSON or text file).

- If the task includes a UI, it must be **clean, usable, and functional** at a professional freelance standard.
- Any visual assets you bundle (icons, fonts, images) must be **commercially free** — Google Fonts, Lucide, Heroicons, Pexels are safe. **Do not use Unsplash content.** When in doubt, use placeholders.

Read the task twice. Note the requirements, the constraints, and any field that feels ambiguous — those ambiguous fields are exactly what you'll need to nail down in Step 1.

5. Step 1 — Prompt Engineering

This is where the hackathon is won or lost. If your prompt is loose, every downstream step suffers — your tests get brittle, the agent makes weird choices, and your Golden Patch won't reproduce.

You are full-stack engineers. You already know how to write code. What you may not have done before is write a prompt that behaves like a **strict specification**. The rest of this section is the toolkit.

5.1 What a Good Prompt Looks Like

Your rewritten prompt is a **freelance brief turned into a structured specification**. It must be readable by both a human reviewer and an AI agent, with zero room for "creative interpretation." It must contain:

- **Clarity** — goal and tech stack stated in unambiguous language.
- **Context** — every piece of information needed for the agent to build a working solution and for tests to verify it.
- **Expected Interface** — every public function, class, file path, CLI command, or HTTP endpoint that an external test will touch.

5.2 The Determinism Rule

Your prompt must act like a **state machine with one execution path**. The AI agent reading it should never have a choice about *what* to build — only about minor stylistic details that don't affect tests.

This means you must aggressively eliminate **non-deterministic language**. Below are the four traps to watch for:

Trap	Forbidden Phrasing	Why It Fails
Optionality	"use React or Vue", "either X or Y"	The agent picks differently each run; tests can't be stable.
Soft Constraints	"should", "preferably", "if possible", "recommended"	The agent will skip the requirement on some runs.
Open-Ended Scope	"etc.", "and others", "similar tools", "relevant libraries"	The agent hallucinates scope.
Permission	"you can", "you may", "feel free to"	States capability instead of instruction.

Replace every one of those with imperative commands: `MUST`, `SHALL`, `Implement`, `Create`, `Use`.
 If a requirement isn't a hard `MUST`, delete it — soft requirements are worse than no requirements.

Determinism in practice

Bad	Good
"You can use Express or Fastify for the backend."	"Use Express 4.x for the backend."
"It is recommended to validate inputs."	"Validate all inputs against the schema in <code>src/schemas/</code> . On failure, return HTTP 400."
"Build the API using relevant Node.js libraries."	"Use Node.js 20, Express 4.x, and <code>zod</code> for validation. No other runtime libraries."
"Allow the user to mark a card as correct or wrong (c/w, y/n, etc.)."	"Allow the user to mark a card by entering <code>c</code> or <code>w</code> (case-insensitive). Any other input re-prompts."

KEYWORD SWEEP

Run a final scan over your prompt. If `or`, `etc.`, `should`, or `you can` appear anywhere in a requirement, fix them.

5.3 The Expected Interface — the heart of the prompt

The Expected Interface is a section that lists **every public surface** an external test will touch. Without it, tests cannot be written reliably and the agent will name things differently every run.

Write one entry per public surface. The schema is fixed:

- Path: <exact file path>
- Name: <Class.method, function name, or endpoint>
- Type: <function | class | method | API endpoint | React Component | CLI command |
- Input: <parameters and types>
- Output: <return type or HTTP response shape>
- Description: <what it does and what tests verify>

Optional language-specific fields, when relevant:

- **TypeScript / Java:** Inheritance: extends <Base>; implements <IfaceA, IfaceB>
- **Python:** Bases / Overrides: bases: <BaseA>; overrides: <Base.method>
- **Go:** Embedding / Implements: embeds <TypeA>; implements <IfaceA>
- **Decorators:** @dataclass, @override, @Inject

RULE OF THUMB

Every test you write must hit something in the Expected Interface. If a test wants to call `parse_deck()`, then `parse_deck` must be in the interface. Do *not* list internal helpers or private methods — the interface is the *minimum public surface*, not the complete file map.

5.4 Prompt Structure — Two Patterns

Pick one and stick to it. Both are battle-tested.

Pattern A — small to medium tasks

```
# Title
## Description / Context
## Tech Stack
## Key Requirements      (with ### subsections by feature)
## Expected Interface    (with ### per public surface)
## Current State
```

Pattern B — larger or multi-component tasks

```
# Title
## Description
## Current State
## Required Implementation
## Expected Interface    (with ## and ### groupings)
## Deliverables / Acceptance Criteria
```

`Current State` is almost always: "*Empty repository with test files only.*" — that's by design. You are building from scratch.

5.5 A Worked Example (Easy)

To make this concrete, here is a small seed brief and the rewritten prompt.

SEED BRIEF

Need a terminal-based flashcard app for medical exam prep. Loads cards from a text file, shuffles them, shows the question, then the answer, then asks if I got it right. Show me my score at the end.

Rewritten prompt

```
# Title
Terminal-Based Flashcard Application

## Description / Context
Build a terminal-based flashcard tool for medical exam preparation. The tool loads a deck from a local text file (question on one line, answer on the next, blank line between cards). It displays the question, waits for Enter to reveal the answer, then lets the user mark each card as correct or wrong. At the end, it shows the session score. The deck MUST be shuffled at the start of every session. All data MUST be bundled in the codebase – no live fetching, no external dataset downloads. The application MUST run fully offline and locally.

## Tech Stack
- Language: Python 3.10+
- Data: Local text file included in the repository
- No external APIs or dataset downloads

## Key Requirements

### Deck File Format
- One question per line
- Answer on the immediately following line
- One or more blank lines between card pairs
- Trailing blank lines at EOF MUST be tolerated
- Lines MUST be stripped of leading/trailing whitespace

### Core Behaviour
1. Load the deck from the file path passed on the command line.
2. Shuffle the deck at the start of every session.
3. For each card: display the question, wait for the user to press Enter, reveal the answer, then accept input `c` (correct) or `w` (wrong). Any other input MUST re-prompt without advancing.
4. At session end, print exactly one line: `Score: <correct>/<total>`.

## Expected Interface

### Deck Loader
- Path: flashcard.py
- Name: load_deck
- Type: Function
- Input: filepath: str
- Output: list[tuple[str, str]]
- Description: Parses the deck file and returns a list of (question, answer) pairs. Raises ValueError on malformed input (odd number of non-blank lines).

### Entry Point
- Path: flashcard.py
- Name: main
- Type: Function
- Input: filepath: str
- Output: None
- Description: Runs the full session: shuffle, iterate, score, and print the
```



```
final score line in the exact format `Score: <correct>/<total>`.
```

CLI Execution

- Path: flashcard.py
- Name: __main__ block
- Type: CLI entry point
- Input: One positional argument – the deck file path
- Output: None (interactive terminal session)
- Description: When invoked as `python flashcard.py <path>`, reads the deck path and calls `main(filepath)`. Exits cleanly when the session ends.

Bundled Deck

- Path: data/sample_deck.txt
- Name: sample_deck.txt
- Type: File
- Description: A bundled deck file with at least 5 cards in the required format. Must be readable by `load\_deck("data/sample\_deck.txt")`.

Current State

Empty repository with test files only.

Notice what this prompt does:

- Every requirement is a **MUST**.
- Every public surface a test would touch is in the Expected Interface.
- The score format is exact (**Score: 5/10**) — so tests can assert on it.
- The keypress is exact (**c** / **w**) — no "or y/n or anything else".
- No room is left for the agent to invent file structure beyond what's specified.

5.6 Final Prompt Checklist

- ☐ Every requirement uses **MUST** / **SHALL** (no **should**, **or**, **etc.**, **you can**).
- ☐ Tech stack lists exact versions where relevant.
- ☐ Every public symbol a test will reference is in the Expected Interface.
- ☐ Output formats (strings, JSON shapes, error types) are specified exactly.
- ☐ No requirement is added that you can't verify with a test.

6. Step 2 — Writing the Test Suite (TDD)

This is **Test-Driven Development**: write the tests *before* you build the Golden Patch. Doing it in this order has two huge benefits — it forces your prompt to be testable, and it prevents the most common failure mode of this kind of task: tests accidentally over-fitted to your own implementation.

6.1 The Three Testing Principles

1. Functional, not static

A test must *execute* the code, not search the source for keywords. If your test passes because the word `month` appears in a comment, your test is broken. Real test runners (pytest, vitest, jest) calling the public interface — that is the bar.

2. Black-box only

Tests interact with the code through the **Expected Interface only** — public functions, CLI commands, HTTP endpoints. Do not patch internal helpers. Do not assert on private method names. Do not check that a specific file exists in `src/utils/` unless the prompt requires it.

3. Fail-to-Pass deterministic

The exact same test suite must:

- **FAIL** every test on an empty codebase
- **PASS** every test once the Golden Patch is applied

IMPORTANT

If a test errors out (e.g. `ImportError`) on an empty codebase, that's not a fail — that's a crash. Crashes don't count. Your tests must fail *cleanly* (assertion errors, missing-attribute errors handled by the test runner).

6.2 The Over-Specificity Trap

Over-specific tests are the single biggest reason a task gets rejected by reviewers. Avoid:

Over-specific (bad)	Implementation-agnostic (good)
<code>assert "express" in open("index.js").read()</code>	<code>response = requests.post("/api/checkout", json=...); assert response.status_code == 200</code>
<code>assert "month" in source_code</code>	<code>result = filter_by_month(data, 3); assert len(result) == 4</code>
<code>mock.patch("src.file_sync.paramiko.SSHClient")</code>	Run the function against a real (or in-memory) target and assert the side effect.
Test fails if the user names the variable <code>uid</code> instead of <code>user_id</code> (when the prompt didn't specify).	Test asserts only on the public return value, regardless of internal naming.

THE ACID TEST

Ask yourself for every test: "If a different developer wrote a different valid solution that satisfies my prompt, would this test still pass?" If the answer is no — fix the test or fix the prompt.

6.3 What to Test

For a backend-heavy task, your suite should cover:

- **Happy path I/O** — for every Expected Interface entry, send a normal input, assert the normal output.
- **Edge cases** — empty inputs, malformed inputs, boundary values.
- **Side effects** — for tasks with file writes, database inserts, or HTTP calls, assert the resulting state, not just the return value.
- **Error behaviour** — when the prompt specifies an exception or status code, assert it explicitly.

For frontend-heavy tasks, lean lightweight: render a component and assert it renders the expected text, prop-driven content, or click handler effect.

6.4 The Baseline Run

Once your tests are written:

1. Build the Docker image (Step 3).
2. Run `bash run.sh` against the **empty codebase** inside the container.
3. Confirm: every test in `before.json` reports `FAILED` (not `ERRORRED`).

If anything is `ERRORRED`, fix the test — it must fail cleanly. This is the *Fail* half of Fail-to-Pass.

7. Step 3 — Docker Environment & Test Runner

Three files set up the runtime: `Dockerfile`, `run.sh`, `parsing.py`. They follow strict templates — the validation script depends on them.

DO NOT CHANGE STRUCTURE

You may add dependencies. You may not change the layout, remove existing instructions, add unrelated layers, or change entry points.

7.1 Dockerfile (template)

```
#####
# BASE IMAGE
#####
FROM ubuntu:22.04
ENV DEBIAN_FRONTEND=noninteractive

#####
# SYSTEM DEPENDENCIES
#####
RUN apt-get update && apt-get install -y \
    git \
    python3 \
    python3-pip \
    python3-setuptools \
    python-is-python3 \
    unzip \
    && rm -rf /var/lib/apt/lists/*

#####
# WORKING DIRECTORY + GIT SETUP
#####
WORKDIR /app
RUN git init \
    && git config --global user.email "agent@example.com" \
    && git config --global user.name "Agent" \
    && echo "# Workspace" > README.md \
    && git add README.md \
    && git commit -m "Initial commit"

#####
# EVALUATION ASSETS DIRECTORY
#####
RUN mkdir -p /eval_assets

CMD ["/bin/bash"]
```

You may add language-specific dependencies in the `SYSTEM DEPENDENCIES` block (e.g. `nodejs`, `npm`, `default-jdk`). Do not modify any other section. Do not add `COPY` commands.

7.2 run.sh (template)

```
#!/bin/bash
### COMMON SETUP; DO NOT MODIFY ###
set -e

# --- CONFIGURE THIS SECTION ---
run_all_tests() {
    echo "Running all tests..."
    # TODO: replace with your test runner
    # Example: pytest tests/ -v
    # Example: npm test
}
# --- END CONFIGURATION SECTION ---

### COMMON EXECUTION; DO NOT MODIFY ###
run_all_tests
```

Replace only the body of `run_all_tests`. Leave the rest alone.

7.3 parsing.py (template)

Implement only `parse_test_output`; do not touch anything outside the marked section.

```

import dataclasses, json, sys
from enum import Enum
from pathlib import Path
from typing import List

class TestStatus(Enum):
    PASSED = 1
    FAILED = 2
    SKIPPED = 3
    ERROR = 4

@dataclasses.dataclass
class TestResult:
    name: str
    status: TestStatus

### DO NOT MODIFY THE CODE ABOVE ###

### Implement the parsing logic below ###
def parse_test_output(stdout_content: str, stderr_content: str) -> List[TestResult]:
    """Parse the test output content and extract test results."""
    raise NotImplementedError('Implement the test output parsing logic')
### Implement the parsing logic above ###

### DO NOT MODIFY THE CODE BELOW ###
def export_to_json(results, output_path):
    json_results = {'tests': [{'name': r.name, 'status': r.status.name} for r in results]}
    with open(output_path, 'w') as f:
        json.dump(json_results, f, indent=2)

def main(stdout_path, stderr_path, output_path):
    with open(stdout_path) as f: stdout = f.read()
    with open(stderr_path) as f: stderr = f.read()
    results = parse_test_output(stdout, stderr)
    export_to_json(results, output_path)

if __name__ == '__main__':
    if len(sys.argv) != 4:
        print('Usage: python parsing.py <stdout_file> <stderr_file> <output_json>')
        sys.exit(1)
    main(Path(sys.argv[1]), Path(sys.argv[2]), Path(sys.argv[3]))

```

7.4 Building & Running

From the project root (where the `Dockerfile` lives):

```

docker build -t my-task .
docker run -it my-task:latest /bin/bash
# Inside the container:
bash run.sh

```

If `bash run.sh` produces nothing or errors, fix `run.sh` and `parsing.py` before going further.

8. Step 4 — Building the Golden Patch

Now and only now — after the prompt and tests are stable — do you build the actual solution.

You can use any coding agent: Cursor, OpenCode, Claude Code, your own setup. Feed your **rewritten prompt** to the agent and let it generate the first iteration. Then iterate until:

1. Every test passes.
2. The Golden Patch satisfies every requirement in your prompt — including ones not directly tested (e.g. "the UI must look professional").
3. No copyrighted images or assets are used.
4. No external API keys, no live network fetches.

Helper functions, third-party libraries, and additional internal files are fine — as long as they don't contradict the prompt. The Golden Patch is a real codebase, not a minimal stub.

WHEN TESTS FAIL

The bug is in **either** the patch **or** the test. If the patch is wrong, fix it. If the test is over-specific or wrong, fix the test, then re-baseline against an empty codebase to make sure the new test still fails cleanly there.

9. Step 5 — Validation Script

The validation script is the final gate. It does the full Fail-to-Pass cycle automatically and produces two files:

- `before.json` — test results against the empty codebase. Every test must be `FAILED`.
- `after.json` — test results against your Golden Patch. Every test must be `PASSED`.

9.1 Folder Layout

`validation.sh` must be run from `/app`, and `/app` must contain exactly:

```
/app/
├── Dockerfile      # your finalized Dockerfile
├── tests.zip       # your test suite
├── codebase.zip    # your Golden Patch
├── run.sh          # your test runner
└── parsing.py      # your parser
```

9.2 ZIP Conventions (read carefully)

This trips up almost everyone:

File	What's inside	How to create it
tests.zip	A tests/ folder at the root (i.e. tests.zip → tests/ → files)	Zip the tests/ folder itself
codebase.zip	The codebase files directly at the root (i.e. codebase.zip → flashcard.py , data/ , etc.)	Zip the contents of the codebase folder, not the folder itself

COMMON MISTAKE
codebase.zip must **not** be nested. codebase.zip → codebase/ → files is wrong.
codebase.zip → files is right.

9.3 What You're Allowed to Change in validation.sh

Exactly two things:

- 1. The root path (default /app) if your local setup uses a different path.
- 2. If the script doesn't run, you may add these two lines:

```
/bin/bash -c 'parse_results stdout.txt stderr.txt before.json' || true
/bin/bash -c 'parse_results stdout.txt stderr.txt after.json' || true
```

Do not change anything else. Watch your coding agent — agents love to "improve" lines they shouldn't touch.

9.4 Success Criteria

After running validation.sh :

- ☐ before.json lists every test as FAILED .
- ☐ after.json lists every test as PASSED .
- ☐ Both files exist in /app/ .

If either condition fails, debug. Don't submit a partial run.

10. Submission — The app/ Folder

Your final submission is the app/ folder in its post-validation state. It should look exactly like this:


```
app/
├── Dockerfile
├── tests.zip
├── codebase.zip
├── run.sh
├── parsing.py
├── validation.sh
├── before.json      # produced by validation script
└── after.json       # produced by validation script
```

Zip the `app/` folder and submit it. That's it.

Pre-flight checklist

- ☐ `before.json` shows all `FAILED`.
- ☐ `after.json` shows all `PASSED`.
- ☐ `codebase.zip` is flat (no `codebase/` parent).
- ☐ `tests.zip` contains a `tests/` parent folder.
- ☐ `Dockerfile` structure is unchanged outside the dependency block.
- ☐ `run.sh` and `parsing.py` are unmodified outside their `CONFIGURE` sections.

11. Tech Stack Compatibility Guide

The Docker base is Ubuntu 22.04. Some technologies fight this environment. If the seed task lets you choose, prefer the right column:

Category	Avoid (high friction)	Recommended (smooth)
Database	PostgreSQL, MongoDB (need running daemons)	SQLite, in-memory, JSON / text files
Test runner	Jest (heavy), Playwright (browser deps)	pytest, vitest
Language	Go, Rust (compile cycle adds complexity)	Python 3.10+, Node 20 (TS / JS)
Frontend	Next.js with full SSR setup if you've never deployed it	Vite + React, or plain SSR-free React

This is guidance, not a rule. If you're already fluent in Postgres and Go, use them. If you're picking on the fly, pick from the right column.

12. FAQ

Do I have to use Docker on my own machine?

Yes. The validation script runs inside Docker. WSL is fine on Windows but not required — Git Bash or PowerShell with Docker Desktop also work.

My test errors with `ImportError` on the empty codebase. Is that a "fail"?

No. `ERRORRED` is not `FAILED`. Restructure your test (e.g. wrap imports in `try/except` inside the test function, or test for `AttributeError` on a missing function) so it fails cleanly with an assertion error.

Can my Golden Patch have helper files not listed in the Expected Interface?

Yes. The Expected Interface is the *public* surface — internal helpers, configs, and additional modules are fine.

My prompt says "the UI should look professional" but I have no test for it. Is that allowed?

Yes — qualitative requirements are still part of the prompt and the Golden Patch must satisfy them. Just don't try to write a unit test for "looks professional"; satisfy it directly in the patch.

Can I clone an existing website's design?

No. Don't clone specific real-world websites. Take inspiration, build your own.

Can I use a paid API or one that needs an API key?

No. Avoid anything that needs a key, runtime config, or external service. Bundled local data only.

Can I add or remove tests after building the Golden Patch?

Yes — but if you change tests, you must re-run the empty-codebase baseline so `before.json` still shows all `FAILED`. The Fail-to-Pass invariant must hold.

What if a requirement is partially testable and partially not?

Test the testable parts. Implement the non-testable parts in the Golden Patch and trust your reviewers to read the code. The prompt should still state both clearly.

Which agent / LLM should I use for the Golden Patch?

Any. Cursor with Claude works well. OpenCode is free. Use what you're fastest with — the agent is a tool, not the deliverable.

13. Appendix — Hard Prompt Examples

The flashcard example in §5.5 is intentionally easy. Real seed tasks are bigger. Below are two harder examples to study.

13.1 Hard Example — Web Backend (REST API + auth + persistence)

SEED BRIEF

I run a small bookstore. I need an API where customers can sign up, log in, browse books, and place orders. Orders should keep the user's history.

Rewritten prompt

Title

Bookstore REST API with Authentication and Order History

Description / Context

Build a REST API for a small bookstore that supports user signup, login, book browsing, and order placement. Each authenticated user MUST be able to view their own order history. The system MUST persist users, books, and orders to a local SQLite database file at `data/store.db`. The API MUST run fully offline. No external services, no external API calls.

Tech Stack

- Language: Node.js 20
- Framework: Express 4.x
- Database: SQLite (via the `better-sqlite3` package)
- Auth: JSON Web Tokens (via `jsonwebtoken`); password hashing via `bcrypt`
- Validation: `zod`
- Test runner: vitest with supertest

Key Requirements

Authentication

1. `POST /auth/signup` – accepts `{ email, password }`. Email MUST be a valid email; password MUST be at least 8 characters. On success, returns HTTP 201 with `{ id, email }`. On duplicate email, returns HTTP 409.
2. `POST /auth/login` – accepts `{ email, password }`. On success, returns HTTP 200 with `{ token }` (JWT signed with HS256, expiry 1 hour, secret read from `JWT\_SECRET` environment variable, default `"dev-secret"`). On invalid credentials, returns HTTP 401.
3. All endpoints below `/orders` MUST require a valid JWT in the `Authorization: Bearer <token>` header. Missing or invalid token returns HTTP 401.

Books

4. `GET /books` – returns HTTP 200 with `[ { id, title, author, price } ]`. Available to anyone (no auth required). The book catalog MUST be pre-seeded from `data/seed\_books.json` on first startup.
5. `GET /books/:id` – returns HTTP 200 with `{ id, title, author, price }`, or HTTP 404 if not found.

Orders

6. `POST /orders` – accepts `{ bookIds: number[] }`. Creates an order owned by the authenticated user. Returns HTTP 201 with `{ id, userId, bookIds, total, createdAt }`. Total is the sum of book prices. Invalid bookIds return HTTP 400.
7. `GET /orders` – returns HTTP 200 with the authenticated user's orders, newest first. Other users' orders MUST NOT appear.

Persistence

8. SQLite schema MUST contain three tables: `users`, `books`, `orders`. Schema MUST be created on startup if missing.
9. Passwords MUST be stored as bcrypt hashes (cost factor 10).

Expected Interface

Signup endpoint

- Path: src/routes/auth.js
- Name: POST /auth/signup
- Type: API Endpoint
- Input: JSON body `{ email: string, password: string }`
- Output: HTTP 201 `{ id: number, email: string }` | HTTP 400 `{ error: string }` | HTTP 409 `{ error: string }`
- Description: Creates a new user. Hashes password with bcrypt before insert. Tests verify happy path, validation, and duplicate email.

Login endpoint

- Path: src/routes/auth.js
- Name: POST /auth/login
- Type: API Endpoint
- Input: JSON body `{ email: string, password: string }`
- Output: HTTP 200 `{ token: string }` | HTTP 401 `{ error: string }`
- Description: Verifies credentials and issues a JWT. Tests verify the JWT is signed with HS256 and decodes to `{ userId, email }`.

List books endpoint

- Path: src/routes/books.js
- Name: GET /books
- Type: API Endpoint
- Input: None
- Output: HTTP 200 `[{ id: number, title: string, author: string, price: number }]`
- Description: Returns the full book catalog. Tests verify the seeded catalog is returned and contains at least 5 books.

Create order endpoint

- Path: src/routes/orders.js
- Name: POST /orders
- Type: API Endpoint
- Input: JSON body `{ bookIds: number[] }`, header `Authorization: Bearer <token>`
- Output: HTTP 201 `{ id, userId, bookIds, total, createdAt }` | HTTP 400 | HTTP 401
- Description: Creates an order for the authenticated user. Tests verify total is computed correctly and that anonymous requests return 401.

List own orders endpoint

- Path: src/routes/orders.js
- Name: GET /orders
- Type: API Endpoint
- Input: header `Authorization: Bearer <token>`
- Output: HTTP 200 `[{ id, bookIds, total, createdAt }]` | HTTP 401
- Description: Returns the authenticated user's orders, newest first. Tests verify isolation: User A's orders do not appear in User B's list.

Database initializer

- Path: src/db.js
- Name: initDb
- Type: Function

```

- Input: dbPath: string
- Output: Database instance
- Description: Opens (or creates) the SQLite database at the given path,
  creates the three tables if absent, seeds books from
  `data/seed_books.json` if the books table is empty.

### Seed data file
- Path: data/seed_books.json
- Name: seed_books.json
- Type: File
- Description: A JSON array of at least 5 books, each with `title`,
  `author`, and `price` (number). Read by `initDb` on first startup.

### Server entry point
- Path: src/server.js
- Name: app
- Type: Express application
- Description: Exports the configured Express app. Tests import this
  module directly with supertest; production code calls
  `app.listen(...)` separately.

## Current State
Empty repository with test files only.

## Required Implementation
- Implement all five endpoints with the exact behaviour above.
- Initialise SQLite at `data/store.db` on first request.
- Seed books from `data/seed_books.json` if the books table is empty.
- Hash passwords with bcrypt (cost 10).
- Sign JWTs with HS256 using `process.env.JWT_SECRET || "dev-secret"`.
- Provide `package.json` with all listed dependencies pinned to specific
  versions.
- Tests MUST be runnable with `npx vitest run`.

```

What makes this prompt hard

- Multi-component (auth + persistence + API + seeding).
- Cross-cutting concern: every order endpoint must enforce auth.
- Stateful: orders persist between requests; user A and user B must be isolated.
- The HTTP contract is exact down to status codes — that gives tests something concrete to assert.
- The DB path, seed file, JWT secret, and bcrypt cost are all specified — zero room for the agent to wiggle.

13.2 Hard Example — Full-stack with React + API + Local Data

SEED BRIEF

Build a small dashboard that shows my company's monthly sales. I want to filter by product category and see a chart.

Rewritten prompt

```
# Title
Monthly Sales Dashboard with Category Filter

## Description / Context
Build a single-page dashboard that visualises monthly sales totals by
product category. The dataset MUST be bundled in the repository as
`data/sales.json`. The dashboard MUST run fully offline.

## Tech Stack
- Frontend: React 18 + Vite, TypeScript
- Charting: Recharts
- Backend: Express 4.x serving `/api/sales`
- Test runner: vitest (frontend), supertest (backend)

## Key Requirements

### Data
1. `data/sales.json` MUST contain at least 24 records. Each record:
  `{ date: "YYYY-MM-DD", category: string, amount: number }`.
2. At least 3 distinct categories MUST appear.

### API
3. `GET /api/sales` – returns the full dataset, HTTP 200, JSON array.
4. `GET /api/sales?category=<name>` – returns only matching records,
  HTTP 200. Unknown category returns HTTP 200 with `[]`.

### Frontend
5. The dashboard MUST render a category filter (`<select>`) populated
  from the categories present in the dataset, plus an `All` option.
6. The dashboard MUST render a Recharts `<BarChart>` showing monthly
  totals (one bar per month) for the selected category (or all).
7. When the filter changes, the chart MUST update without a full reload.
8. The currently selected category MUST be visible as plain text inside
  `<p data-testid="selected-category">Showing: <name></p>` for testability.

## Expected Interface

### API endpoint
- Path: server/index.ts
- Name: GET /api/sales
- Type: API Endpoint
- Input: optional query param `category: string`
- Output: HTTP 200 with `Sale[]`
- Description: Reads `data/sales.json` and filters by category if
  provided. Tests assert: full list returned without filter; correct
  filter behaviour with category; empty array for unknown category.

### Dashboard component
- Path: src/Dashboard.tsx
- Name: Dashboard
- Type: React Component
- Input: props: `{ initialState: Sale[] }`
```

```
- Output: JSX
- Description: Renders the ``, the ``, and a
  `

\geq 3 categories.
- The dev server MUST start with `npm run dev` and proxy `/api` to the
  Express server.
- Tests MUST be runnable with `npm test`.


```

What makes this prompt hard

- Crosses front-end and back-end, each with its own test approach.
- "When the filter changes, the chart MUST update" is testable only because of the `data-testid="selected-category"` requirement — that hook is what makes the qualitative requirement testable. Adding test-friendly hooks like this in your prompt is one of the most useful skills in this hackathon.
- Specific charting library (Recharts) — not "a chart library."
- Bundle requirements are explicit (24 records, 3 categories) — tests can rely on the dataset shape.

Final Word

The whole point of this hackathon is the loop:

prompt → tests → solution → validation

You will not get the loop right on the first try. That's expected. The fastest way to a strong submission is to walk the loop end-to-end *quickly* — even with a small task — and then tighten each step. Don't spend two hours perfecting your prompt only to find out at minute 150 that your test runner doesn't fit the Docker base.

When you're stuck, the answer is almost always: **"is my prompt deterministic?"** If it is, the rest follows. If it isn't, nothing will.

Good luck. Build something verifiable.